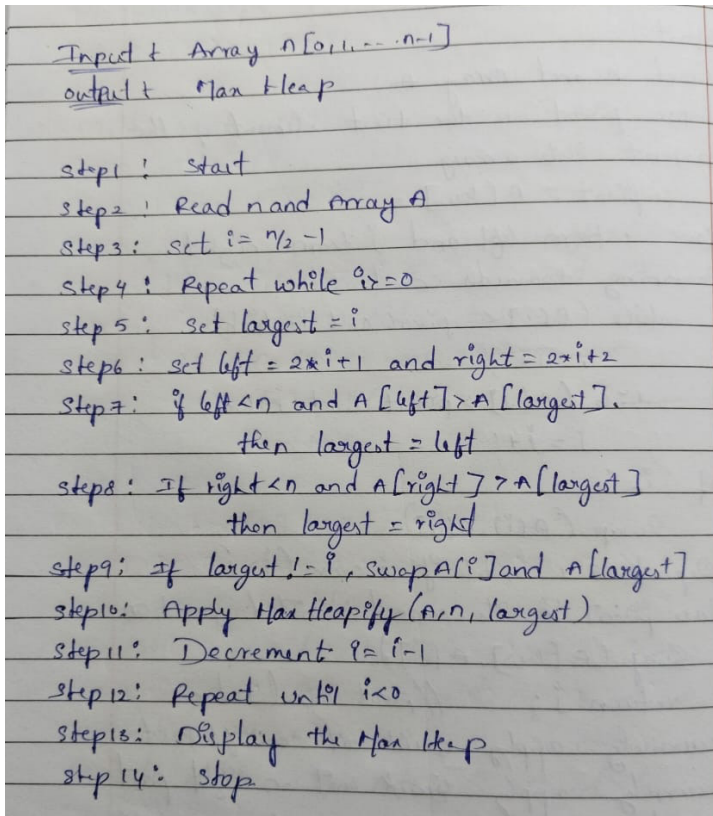


Practical - 3

Aim: Implementation of max-heap sort algorithm

Max-heap sort

Algorithm:



Input: Array $A[0, 1, \dots, n-1]$
Output: Max Heap

Step 1: Start
Step 2: Read n and Array A
Step 3: Set $i = n/2 - 1$
Step 4: Repeat while $i \geq 0$
Step 5: Set $\text{largest} = i$
Step 6: Set $\text{left} = 2*i + 1$ and $\text{right} = 2*i + 2$
Step 7: If $\text{left} < n$ and $A[\text{left}] > A[\text{largest}]$,
then $\text{largest} = \text{left}$
Step 8: If $\text{right} < n$ and $A[\text{right}] > A[\text{largest}]$,
then $\text{largest} = \text{right}$
Step 9: If $\text{largest} \neq i$, swap $A[i]$ and $A[\text{largest}]$
Step 10: Apply MaxHeapify($A, n, \text{largest}$)
Step 11: Decrement $i = i - 1$
Step 12: Repeat until $i < 0$
Step 13: Display the Max Heap
Step 14: Stop

Program:

```
#include <iostream>
using namespace std;
class MaxHeap
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
};
```



```
void printArray()
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    cout << endl;
}

void maxheapify(int arr[], int n, int i)
{
    int largest = i; int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) { swap(arr[i],
        arr[largest]); maxheapify(arr,
        n, largest);
    }
}

void heapsort() { for (int i = n / 2
- 1; i >= 0; i--) { maxheapify(arr,
n, i);

    }
    for (int i = n - 1; i > 0; i--)
        { swap(arr[0], arr[i]);
        maxheapify(arr, i, 0);
    }
}

int main()
{
    MaxHeap mh;
```

```
mh.arrayInput();

cout << "\nBefore Sorting: ";
mh.printArray();

mh.heapsort();

cout << "After Sorting: ";
mh.printArray();

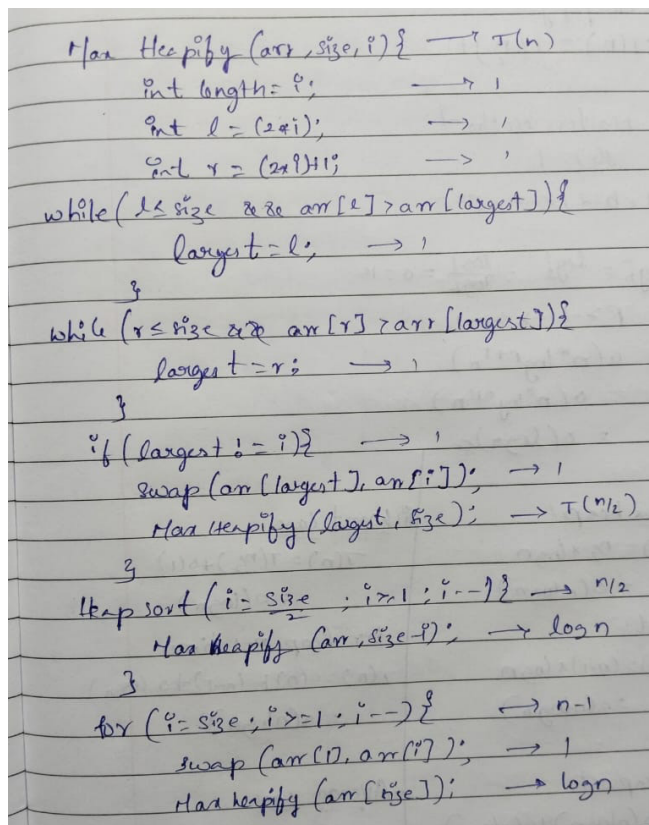
return 0; }
```

Output:

```
lokeshgandreti@LOKESHs-MacBook-Air practical-3 % cd "/Users/lokeshgandr
eti/Desktop/c++/practical-3/" && g++ maxheap.cpp -o maxheap && "/Users/
lokeshgandreti/Desktop/c++/practical-3/"maxheap
Enter number of elements: 5
Enter elements: 10 3 8 6 15

Before Sorting: 10 3 8 6 15
After Sorting: 3 6 8 10 15 %
lokeshgandreti@LOKESHs-MacBook-Air practical-3 %
```

Time Complexity:



$\text{Max Heapify}(\text{arr}, \text{size}, i) \rightarrow T(n)$
 $\text{int length} = i; \rightarrow 1$
 $\text{int l} = (2*i); \rightarrow 1$
 $\text{int r} = (2*i+1); \rightarrow 1$
 $\text{while} (l \leq \text{size} \ \&\& \ \text{arr}[l] > \text{arr}[\text{largest}]) \{$
 $\quad \text{largest} = l; \rightarrow 1$
 $\}$
 $\text{while} (r \leq \text{size} \ \&\& \ \text{arr}[r] > \text{arr}[\text{largest}]) \{$
 $\quad \text{largest} = r; \rightarrow 1$
 $\}$
 $\text{if} (\text{largest} \neq i) \{ \rightarrow 1$
 $\quad \text{swap}(\text{arr}[\text{largest}], \text{arr}[i]); \rightarrow 1$
 $\quad \text{Max Heapify}(\text{largest}, \text{size}); \rightarrow T(n/2)$
 $\}$
 $\text{Heapsort} (i = \frac{\text{size}}{2}; i > 1; i--) \{ \rightarrow n/2$
 $\quad \text{Max Heapify}(\text{arr}, \text{size}-i); \rightarrow \log n$
 $\}$
 $\text{for} (i = \text{size}; i >= 1; i--) \{ \rightarrow n-1$
 $\quad \text{swap}(\text{arr}[0], \text{arr}[i]); \rightarrow 1$
 $\quad \text{Max Heapify}(\text{arr}, \text{size}); \rightarrow \log n$
 $\}$



for Max Heapify
 $T(n) = T(\frac{n}{2}) + 1$

using Master's Method
 $f(n) = 1$
 $a=1, b=2, k=0, p=0$

$\log_b a = \log_2 1 = \frac{\log 1}{\log 2} = 0 = k$
 $p > -1 \Rightarrow 0 > -1$

$T(n) = O(n^k \log^{p+1} n)$
 $= O(n^0 \log^{0+1} n)$
 $= O(\log n)$

<u>Build Max heap</u> $T(n) = n \times \log n$ $= O(n \log n)$ <u>heap sort</u> $T(n) = (n-1) \times \log n$ $= O(n \log n)$ <u>Overall Heapsort</u> $T(n) = O(n \log n) + O(n \log n)$ $= O(n \log n)$ <u>Best case</u> $T(n) = O(n \log n)$	<u>Worst case</u> $T(n) = T(\frac{n}{2}) + O(1)$ $= O(\log n)$ for loop $n-1$ times $T(n) = O(n) + (n-1) \times O(\log n)$ $= O(n \log n)$ <u>Average case</u> same as worst case $T(n) = O(n \log n)$
---	---

Conclusion:

The Max-Heap Sort algorithm was successfully implemented using the Max-Heapify method. The largest element is repeatedly placed at the end of the array, resulting in a sorted array in ascending order. The time complexity of Heap Sort is $O(n \log n)$.